Lesson 11: Advanced Data Visualization

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Static vs interactive — when to use which?
I Do	10–15 min	Pandas plots, Plotly, Streamlit basics
We Do	15–20 min	Build a Streamlit dashboard together
You Do	5–10 min	Add an interactive widget
Wrap-Up	5 min	Final project checklist & presentations

Warm-Up (5 min)

When Would You Use Each?

Scenario	Static or Interactive?
A report for your boss (PDF)	???
An exploration tool for a data team	???
A chart in a research paper	???
A live dashboard for monitoring sales	???

Share your answers in the chat.

I Do: Pandas Built-In Plotting

Pandas can create quick plots directly from DataFrames:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [100, 150, 200, 250, 300, 350],
    "Expenses": [80, 120, 180, 200, 220, 300]
}
df = pd.DataFrame(data)

df.plot(x="Month", y=["Sales", "Expenses"],
        kind="line", title="Sales vs. Expenses")
plt.show()
```

I Do: Pandas Built-In Plotting (Cont.)

Supported `kind` values: `line`, `bar`, `barh`, `hist`, `scatter`, `pie`

> Pandas plotting is great for quick exploration. For polish, use Plotly or Seaborn.

I Do: Plotly — Interactive Charts

Plotly creates interactive charts with hover, zoom, and pan:

```
import plotly.express as px
import plotly.data as pldata

df = pldata.iris(return_type="pandas")
fig = px.scatter(df, x="sepal_length", y="petal_length",
                 color="species",
                 title="Iris: Sepal vs Petal Length",
                 hover_data=["petal_length"])
fig.write_html("iris.html", auto_open=True)
```

I Do: Plotly — Interactive Charts (Cont.)

Key Plotly features:

- Hover tooltips show data values
- Click and drag to zoom
- Double-click to reset
- Save as PNG from the toolbar

> Don't use `fig.show()` — it can hang. Use `fig.write_html()` instead.

I Do: What Is Streamlit?

Streamlit turns Python scripts into web apps:

```
import streamlit as st

st.title("My First App")
st.write("Hello, world!")
```

Run it:

```
streamlit run app.py
```

Opens at: `http://localhost:8501`

Why Streamlit?

- Write Python → get a web app
- No HTML, CSS, or JavaScript needed

I Do: Streamlit Components

Text and display:

```
st.title("Big Title")
st.header("Section Header")
st.subheader("Subsection")
st.text("Plain text")
st.markdown("**Bold** and *italic*")
st.code("print('hello')", language="python")
```

Data display:

```
st.write(df)              # Auto-renders DataFrames
st.dataframe(df)          # Interactive table
st.metric("Sales", "$500", "+10%")  # KPI card
```

> `st.write()` is the Swiss Army knife — it handles strings, DataFrames, charts, and more.

I Do: Streamlit Input Widgets

Capture user input for interactive filtering:

```
# Text and numbers
name = st.text_input("Your name", "John")
age = st.slider("Age", 0, 100, 25)

# Selection
option = st.selectbox("Choose one", ["A", "B", "C"])
options = st.multiselect("Choose many", ["X", "Y", "Z"])

# Date and toggles
date = st.date_input("Pick a date")
show = st.checkbox("Show details")
if show:
    st.write("Here are the details!")
```

Important: Every time a widget value changes, Streamlit **reruns the entire script** from top to bottom with the new values.

I Do: Streamlit Layout

Columns — side by side:

```
col1, col2 = st.columns(2)
with col1:
    st.header("Left")
    st.write("Content here")
with col2:
    st.header("Right")
    st.write("More content")
```

I Do: Streamlit Layout (Cont.)

Sidebar — persistent navigation:

```
st.sidebar.title("Filters")
product = st.sidebar.selectbox("Product", df["Product"])
```

Expander — collapsible sections:

```
with st.expander("Click to see details"):
    st.write("Hidden until expanded")
```

> Use the sidebar for filters. Use columns for metrics. Use expanders for optional detail.

I Do: Streamlit + Plotly

Plotly charts render beautifully in Streamlit:

```
import streamlit as st
import plotly.express as px

fig = px.bar(df, x="Product", y=["Sales", "Profit"],
             barmode="group",
             title="Sales and Profit by Product")
st.plotly_chart(fig)
```

Other chart options:

```
st.line_chart(df)         # Quick Streamlit line chart
st.bar_chart(df)          # Quick Streamlit bar chart
st.pyplot(fig)            # Embed a Matplotlib figure
st.plotly_chart(fig)      # Embed a Plotly figure
```

> For the capstone dashboard, use `st.plotly_chart()` for interactive charts and

We Do: Build a Product Dashboard

Let's build a simple dashboard together — step by step.

Step 1: Create `dashboard_app.py`:

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px

# Sample data
np.random.seed(42)
df = pd.DataFrame({
    "Product": ["Product A", "Product B", "Product C", "Product D"],
    "Sales": np.random.randint(100, 500, size=4),
    "Profit": np.random.randint(20, 100, size=4)
})

st.title("Product Dashboard")
```


Run it: `streamlit run dashboard_app.py`

We Do: Add Sidebar Filters

Step 2: Add a product filter in the sidebar:

```
# Sidebar
st.sidebar.header("Filter Options")
selected = st.sidebar.selectbox("Select Product", df["Product"])

# Filter the data
filtered = df[df["Product"] == selected]
```

We Do: Add Sidebar Filters (Cont.)

Step 3: Show metrics for the selected product:

```
col1, col2 = st.columns(2)
with col1:
    st.metric("Sales", f"${filtered['Sales'].values[0]:,}")
with col2:
    st.metric("Profit", f"${filtered['Profit'].values[0]:,}")
```

> Refresh the browser — select different products and watch the metrics update.

We Do: Add a Chart

Step 4: Add a comparison bar chart:

```
st.subheader("Sales and Profit Comparison")
fig = px.bar(df, x="Product", y=["Sales", "Profit"],
             barmode="group")
st.plotly_chart(fig)
```

Step 5: Add a data table:

```
with st.expander("View Raw Data"):
    st.dataframe(df)
```

That's a complete dashboard!

- Sidebar filter for product selection
- Metric cards for key numbers
- Interactive bar chart for comparison

We Do: The Streamlit Pattern

Every Streamlit app follows this pattern:

```
1. Import libraries
2. Load data (from CSV, SQL, or API)
3. Create sidebar filters
4. Filter the data based on user selections
5. Display metrics and charts
6. Add layout (columns, expanders, tabs)
```

```python
# The core loop:
data = load_data()                          # Step 2
filter_value = st.sidebar.selectbox() # Step 3
filtered = data[data["col"] == filter_value]  # Step 4
st.metric("Label", filtered["col"].sum())     # Step 5
st.plotly_chart(px.bar(filtered, ...))        # Step 5
```

> Your capstone dashboard follows this exact pattern — just with real data.

You Do: Add an Interactive Widget (5 min)

Starting from the dashboard we built, add ONE of these:

1. A **slider** that filters products by minimum Sales value
   Hint: `min_sales = st.sidebar.slider("Min Sales", 0, 500, 100)`
2. A **multiselect** that lets users pick multiple products
   Hint: `selected = st.sidebar.multiselect("Products", df["Product"])`
3. A **checkbox** that toggles between showing Sales or Profit
   Hint: `show_profit = st.checkbox("Show Profit")`

Deploying to Streamlit Cloud

Your capstone must be deployed to Streamlit Community Cloud:

1. Push your code to GitHub (include `requirements.txt`)
2. Go to streamlit.io/cloud → log in
3. Click **New App** → select your repo and branch
4. Set the main file path (e.g., `streamlit_app.py`)
5. Click **Deploy**

`requirements.txt` must include:

```
numpy
pandas
matplotlib
plotly
seaborn
streamlit
```

Assignment Preview

Task	What You'll Do
1	Pandas bar chart: employee revenue from SQL
2	Pandas line plot: cumulative revenue
3	Plotly interactive scatter: wind dataset
4	Reflection: static vs interactive visualization
5	Commit your work
6	**Streamlit dashboard for capstone project!**

Assignment Tips

Task 1 (Pandas + SQL): The SQL query is provided in the assignment — just load it with `pd.read_sql_query()`, then use `df.plot(kind="bar")`.

Task 2 (cumulative): Use `df["total_price"].cumsum()` to calculate cumulative revenue.

Task 3 (Plotly wind): The `strength` column is a string — clean it with `str.replace()` and convert to float before plotting.

Task 6 (capstone dashboard):

- Connect to your SQLite database
- Add at least 3 visualizations with `st.plotly_chart()`
- Include user interactions (dropdowns, sliders)
- Deploy to Streamlit Cloud

Final Project Checklist

Kaggle Data Pipeline (Capstone 1)

- [ ] Data loading and inspection
- [ ] Data cleaning and validation
- [ ] Wrangling, aggregation, 2+ engineered features
- [ ] 3+ visualizations with explanations
- [ ] 3+ conclusions supported by charts

Web Scraping Dashboard (Capstone 2)

- [ ] Selenium web scraping → CSV
- [ ] Data cleaning and transformation
- [ ] SQLite database storage
- [ ] Streamlit dashboard with 3+ visualizations

Project Presentations

Record a 3–5 minute video showing both projects:

1. Demo your Kaggle data visualizations
2. Demo your Streamlit web scraping dashboard
3. Describe what you built and what you learned

Recording options:

- Zoom (record a personal meeting with screen share)
- Built-in screen recording (Mac or Windows)
- Loom (online tool)

Upload to YouTube (unlisted) and submit the link.

Wrap-Up

Today we covered:

- Pandas built-in plotting for quick visualizations
- Plotly for interactive charts (hover, zoom, export)
- Streamlit: components, widgets, layout, and deployment
- Building a complete dashboard from data to deploy

You've come incredibly far in this course:

Python basics → Data structures → Pandas → Visualization → Web scraping → SQL → Dashboards

Getting Help This Final Week

- **Streamlit docs:** docs.streamlit.io
- **Plotly docs:** plotly.com/python
- **Streamlit cheat sheet:** cheat-sheet.streamlit.app
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** for deployment help
- **You've got this!**

Congratulations! 🎓

From "Hello, World" to interactive dashboards.

You did it. Be proud of how far you've come.

Thank you for an amazing course!